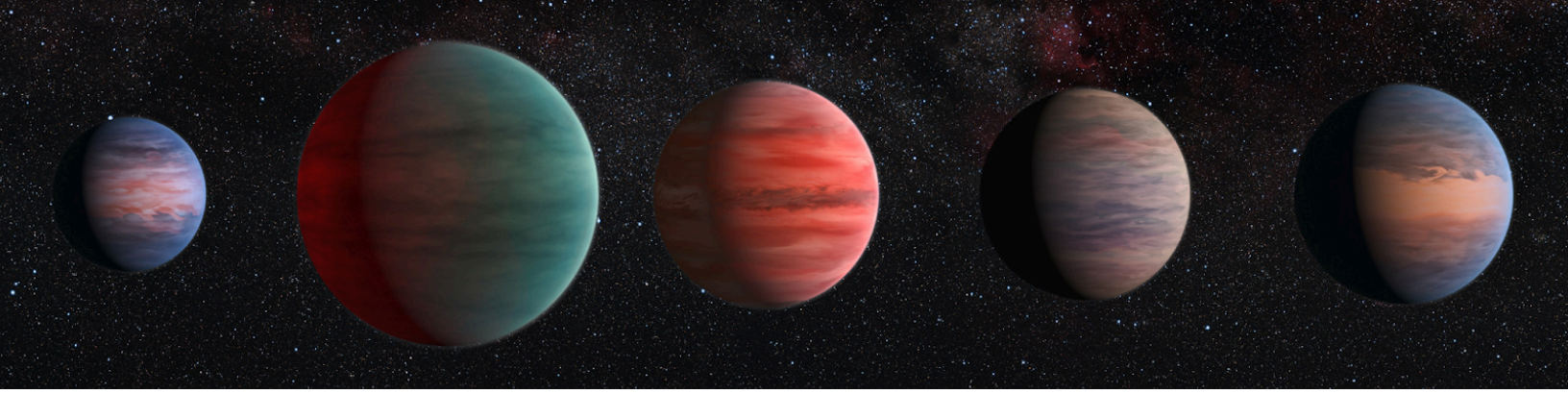


A Data-Driven Approach to Habitability Assessment of Exoplanets

Moorestown Friends School Astronomy Club
Alan Gu

1. Overview

Extrasolar planets, or exoplanets, are planets that exist outside of our own solar system. The study of exoplanets is crucial to understanding planetary formation, contextualizing Earth's features on a cosmic scale, and potentially discovering life outside of Earth. This research class details the assessment of exoplanet habitability, whether or not an exoplanet can sustain life like Earth. For this research class, we will be utilizing the Earth Similarity Index (ESI) to assess an exoplanet planet's habitability by comparing its properties to that of ours. While use of the Planetary Habitability Index (PHI) can be promising, the exoplanet information required by it that we have thus far is not detailed enough to do so.

Scientists can detect exoplanets through a variety of methods depending on the distance and characteristics of the exoplanet, with each method having its limitations. This idea also extends to the assessment of exoplanet properties and habitability–in some cases, like with great distances, some analysis techniques, such as direct imaging, are rendered ineffective, leading to holes within the data. The presence of missing data leads to problems where there is a lack of standardization within habitability assessment. In this class, we use a data-driven approach to the assessment of exoplanet habitability, utilizing calculated approximations of properties wherever there are holes in the data and indicating so in our habitability assessment.

2. Lesson Structure

We place an emphasis on hands-on learning for this class. As opposed to topic-based lessons, we will be adopting a focused lesson structure that tailors to processes involved in actual research projects. We do this to allow students to be able to immediately apply and utilize newly learned information. **The only thing we expect students to have is basic high school STEM knowledge.**

Throughout this class, students will be able to learn and develop the following skills:

- Navigating very commonly used platforms/websites in the research world (physics, finance, AI, engineering, etc.)
- Handling large amounts of data, including missing data
- Understanding professional research workflows and pipelines
- Python and general computer science knowledge
- Various vital research skills like project-designing and management, and research paper literacy, etc.

Students will be learning and using the following tools for research and data analysis:

- Google Colab (Python)
- Python Libraries:
    - Numpy
    - Matplotlib
    - Astropy
- Digital Repositories:
    - Open Exoplanet Catalogue Repository
    - Astrophysics Data System
    - arXiv

The goal of our research class is to introduce students potentially interested in Astronomy to professional Astronomical research in a way that's accessible, interesting, and easy to follow, studying a topic that has grounding real-world applications.

3. Lesson Table of Contents

Proposed Next Lessons:

- Constraining the Methodology
- Finalizing a Research Plan
- Structuring the Coding Project
- Project Setup
- Experimentation & Analysis
- Conclusions & Writeups

1. Introducing Habitability & Finding Tools to Use

Habitability Studies

What is habitability? More specifically, which factors make a given planet habitable and able to support life? Many habitability studies focus on finding environments that could potentially lead to the development of life similar to that of Earth's–the most crucial component being the presence of liquid water, which is essential in a majority of biological processes on our planet.

The most well-known method of doing this is finding a planet in a star's "goldilocks zone". That is, a distance from a given star such that liquid water can exist. Other factors involved include but are not limited to:

- Planet Size & Gravity
- Orbital Eccentricity
- Magnetic Field Presence
- Atmosphere

We will dive deeper into the specifics later, but the main takeaway here is that the habitability of exoplanets is conventionally assessed by comparing an exoplanet's properties to that of Earth's.

Research Tools and Projects

Every research project starts with an idea. The problem is, good ideas are most of the time hard to come by. It is also important to understand what a good research idea is.

A GOOD RESEARCH IDEA IS:

- Feasible given tools, restrictions, and time constraints
- Grounded in theory/principle/logic
- Methodologically and analytically consistent with research practices
- Focused, both topic-wise and goal-wise

So, seeing all these constraints, how do you come up with a good research idea?

Most researchers conduct a **literature review**. That is, they consult and assess academic papers, resources, and knowledge in order to build a foundation for their projects. Now, the logical next question is, "how do you conduct a literature review?".

The internet has lots of opportunities for literature review. One of the most accessible ways for students and researchers to begin their literature review is by consulting **repositories,** essentially storage units on the internet (think of them as public libraries). Repositories can range from collections of research papers to actual research tools.

Research paper repositories used in astronomy include:

- arXiv
- ADSABS (astrophysics data system)
- Inspire HEP

APPLY KNOWLEDGE:

- **Using one of the repositories listed above, find one research paper related to exoplanet habitability, and summarize its abstract (paper summary at the beginning)**

In terms of research tools, **GitHub** is the most popular website for both creating and discovering repositories containing tools.

In order to make sure a project is reasonable, researchers tend to define a set of tools to use to constrain the search for a research idea. In our case, we will be using the **Open Exoplanet Catalogue (OEC)** on GitHub to access information regarding exoplanets. This restricts our research idea to using data within said catalogue, following their data format, and abiding by their data restrictions

Coding Projects

Computer Science and Data Science have almost merged as fields as of recent years. The use of code is especially important within Astrophysics data analyses, where large amounts of information need to be handled.

In order to begin coding, one must first choose a coding language. Coding languages provide ways for coders to interact and give commands to their computer to execute. The most prevalent coding language in modern data analysis is **Python**, known for its easy-to-navigate interface and accessible tools.

Researchers tend to use **integrated development environments** (IDEs) to carry out their code. IDEs are specialized interfaces (text-editing) that allow researchers to type and execute code. Essentially, they make coding a lot more accessible and easy.

Examples Include:

- PyCharm
- Visual Studio Code

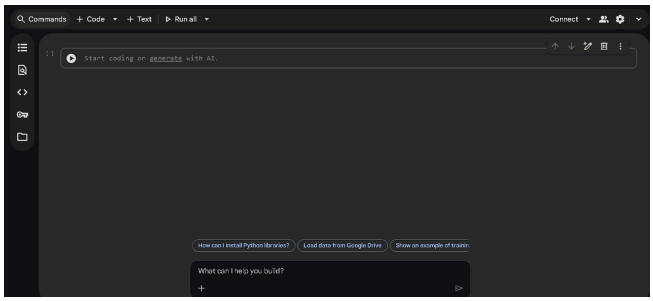
While their code is in development, most researchers don't type full scripts within IDEs; rather, they use a **notebook**, a specialized tool that allows them to break code into cells (individual chunks), that run individually. This allows for them to quickly modify, type, and run code without having to run a full script.

Examples Include:

- Jupyter Notebook
- Google Colab

Putting Everything Together

We will be using Google Colab for this project. Google Colab is an online notebook that allows users to execute Python code, and setup is almost instant. Head over to https://colab.research.google.com/ and click the New Notebook button. Your interface should look something like this:



The textbook you see at the top is a cell. Click on it and type the following code into it, and run it by holding shift and pressing enter:

```
print("Hello World!")
```

print() is a Python command that tells the machine to return whatever message is entered in the parentheses to the user. In this case, it's "Hello World!".

Notice how the quotation marks didn't show when it was printed. That's because "Hello World!" is a string (indicated by quotation marks), telling the computer that Hello World! is a string of letters. If you did not indicate so, your code would return an error.

Strings are one of the many data types in coding. We will dive deeper into each during our analysis

ASSIGNMENTS:

- If you have not done so already, find an exoplanet habitability-related research paper on a research paper repository, read its Abstract section, and write a summary
- Learn the definitions of the most basic types of data types in Python:
    - Strings
    - Boolean Values
    - Integers vs. Floats
- Find 3 simple Python commands aside from print, and summarize what they do on the most basic level

2. Accessing Exoplanet Data

The exoplanet data we wish to access is located within a GitHub repository called the Open Exoplanet Catalogue. For this lesson, let's try accessing the data and get it in a format usable for data analysis.

STEP 1: SEARCH UP "OPEN EXOPLANET CATALOGUE GITHUB" TO FIND THE REPOSITORY

A common standard for GitHub repositories is the inclusion of a README.md text file that provides instructions on how to use the given tools. This repository is licensed under the MIT license, which allows for the contents in it to be used without any copyright restrictions.

Luckily, our README file provides instructions on how to access the data within the catalogue in Python.

```
import xml.etree.ElementTree as ET, urllib.request, gzip, io
url =
"https://github.com/OpenExoplanetCatalogue/oec_gzip/raw/master/systems.xml.gz"
oec =
ET.parse(gzip.GzipFile(fileobj=io.BytesIO(urllib.request.urlopen(url).read())))

# Output mass and radius of all planets
for planet in oec.findall(".//planet"):
    print([planet.findtext("mass"), planet.findtext("radius")])

# Find all circumbinary planets
for planet in oec.findall(".//binary/planet"):
    print(planet.findtext("name"))

# Output distance to planetary system (in pc, if known) and number of planets
in system
for system in oec.findall(".//system"):
    print(system.findtext("distance"), len(system.findall(".//planet")))
```

As starters, let's break each part of the code down and learn the Python behind it along the way.

This code covers a big portion of the Python fundamentals, so this will be useful for us.

1.

```
import xml.etree.ElementTree as ET, urllib.request, gzip, io
```

In Python, we can access external libraries that contain pre-written code (modules) that we can import and use for ourselves. This is one of the biggest reasons why Python is gaining popularity–data analysis is much less of a chore with outside help.

We do so via the import command. In this case, we're importing xml.etree.ElementTree, which is a module that allows us to read the compressed Exoplanet data files. We add an "as ET" afterwards so we can just call the script via ET rather than typing out the whole thing again.

We can import multiple libraries/modules in one line by separating them with a comma. In this case, we're also importing the urllib.request module for accessing online data, gzip for accessing the compressed exoplanet zip files, and io for data processing.

2.

```
url =
"https://github.com/OpenExoplanetCatalogue/oec_gzip/raw/master/systems.xml.gz"
```

In Python, we can define what we call variables by just setting a name equivalent to a given value. This relates back to our "data types" discussion in the previous class. Instead of retyping an entire url, we can just call upon the variable "url". On top of that, if we change what url equals, then the entire code changes, which is incredibly useful for data analysis if we want to add or change data that's used multiple times throughout the code.

3.

```
oec =
ET.parse(gzip.GzipFile(fileobj=io.BytesIO(urllib.request.urlopen(url).read())))
```

Here we access the Open Exoplanet Catalogue and set it as a variable, oec. We use all our modules in order to convert the file into a format that's readable by us.

4.

```
# Output mass and radius of all planets
```

```
for planet in oec.findall(".//planet"):
    print([planet.findtext("mass"), planet.findtext("radius")])
```

The hashtag symbol allows us to create text annotations that the computer doesn't read. Afterwards, we run a for loop, which basically allows us to repeatedly execute a command that works on each individual exoplanet within the catalog. We can access the exoplanets via our new oec's findall property, setting ".//planet" as our data type of desire.

Our for loop allows us to define a variable for each exoplanet that's passed through the loop. In this case, it's just planet. Within the for loop, we print an array (a collection of numbers [1, 2, 3]) containing the mass and radius of each exoplanet. Each instance (planet) within the file has an attribute findtext which we can call to obtain a specific property of desire, typed within a string "".

5.

```
# Find all circumbinary planets
for planet in oec.findall(".//binary/planet"):
    print(planet.findtext("name"))
```

This code is of the same principles, except it's only finding the name of each binary planet and it's looking within the binary planets section for planets orbiting two stars.

6.

```
# Output distance to planetary system (in pc, if known) and number of planets in system
for system in oec.findall(".//system"):
    print(system.findtext("distance"), len(system.findall(".//planet")))
```

This code has the same concepts except it accesses full planetary system data as opposed to singular planet data. It also utilizes the len() command which outputs the number of instances within a given set of data.

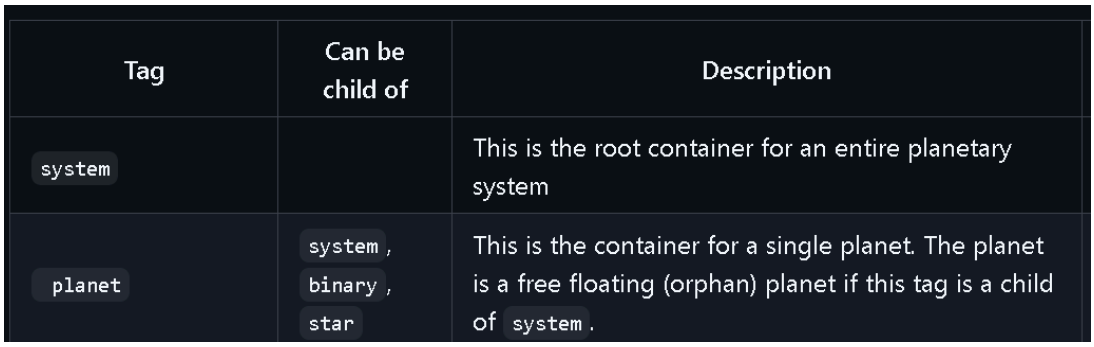
CONCEPT CHECK: If we wish to find earth-like planets within the catalog, which one of these data-fetching loops would be most important for us? (1, 2, 3)

ANSWER: Loop 1, which loops through individual exoplanets would be most useful. We can compare individual exoplanet properties to that of Earth's

Open up Google Colab and go back to the notebook you started. Paste in your imports, the variables set, and the first loop within the repository's tutorial code into a cell, and run it. Take

note of what's printed, and try to identify what each value and array within the printed data represents.

ASSIGNMENTS:

- Answer the following questions:
  - What does each individual array printed by the loops represent?
  - If you had 5000 total exoplanet systems within the catalog, would you have under, over, or exactly 5000 exoplanets within the catalog?
  - Using your answers to the questions above, how many arrays would the first loop print if you ran it in the given scenario?
- A data-accessing tutorial is not the only thing the README GitHub file contains. It also contains the names of properties of objects within the catalog you can access:
  - 

Tag	Can be child of	Description
`system`		This is the root container for an entire planetary system
`planet`	`system`, `binary`, `star`	This is the container for a single planet. The planet is a free floating (orphan) planet if this tag is a child of `system`.

  - The tag refers to what you should type within a string to access the object (in our code it was "mass" or "radius"), and the Can be child of refers to the type of object that can have said tag
- Using the property data table above, modify the code that was provided to you such that it prints properties that were not in the original code.

3. How Can You Begin Data Analysis?

Data analyses can be confusing, time consuming, and difficult to navigate without a clear plan in mind from the beginning. In the next few lessons, we will work to develop a research plan to refer to throughout the course. Experimental Design is a vital component of all research.

Defining Goals

Every great course of action requires a clear-cut list of goals or overarching objectives to function. Before we develop our plan, we must first understand what we're looking to accomplish.

OPEN-ENDED QUESTION: Using your knowledge of exoplanet habitability assessments, what do you think we should be looking for in our data analysis?

To reduce confusion and get a sense of where we should be heading with this analysis, we can consult resources/literature in order to get inspiration. These can range from repositories to papers to even websites. No matter what we use, however, the sources should be credible–that is, they should either provide credible resources, be peer-reviewed, have a lot of citations, or have a clear, understandable walkthrough through their procedure.

For this project, we will use the following website as our source, which provides us all we need for data analysis:

https://phl.upr.edu/projects/earth-similarity-index-esi

Credibility Assessment:

- Cites 2 well-known papers
- Referenced as a "more information" page by the official NASA website
- Equations are well in-line with other literature
- Edu website domain

This website describes the Earth Similarity Index (ESI), which is a scale from 0 (not like Earth at all) to 1 (Earth) that rates exoplanets based on how physically similar they are to Earth. While it's not a direct assessment of a planet's habitability, it serves as a critical component to habitability assessment, as Earth-like exoplanets, from our knowledge, have the best chance of sustaining life.

Moreover, the paper the website cites also suggests that the ESI is to be used in combination with the Planetary Habitability Index (PHI), another scale for directly assessing a

planet's habitability. However, the information required to perform the PHI that we have on exoplanets so far is not detailed enough. The PHI relies on factors that either we don't have any data on (e.g. atmosphere), or we don't have enough information to reliably provide estimates/approximations of said data. The PHI poses an issue that doesn't just include missing information–rather, the data available to use it is not sufficiently detailed enough to conduct quality analyses.

The website provides 2 equations, both which work but focus on different properties of an exoplanet system:

$$1.\ ESI(S,\ R)\ =\ 1\ -\ \sqrt{\frac{1}{2}\left[\left(\frac{S-S_{earth}}{S+S_{earth}}\right)^2\ +\ \left(\frac{R-R_{earth}}{R+R_{earth}}\right)^2\right]}$$

Where S denotes flux (either solar for Earth or stellar for the exoplanet), and R denotes the radius of the corresponding planet.

$$2.\ ESI\ =\ \prod_{i=1}^{n}\left(1\ -\ \left|\frac{x_i-x_{io}}{x_i+x_{io}}\right|\right)^{\frac{w_i}{n}}$$

Where x_i denotes a planetary property, x_{io} is said property for Earth, w_i is a weighting for a given property, and n denotes the number of properties used for the ESI assessment

This is A LOT. Let's break it down.

Let's first define what we got from this on a surface level:

1. We'll be using ESI and following the procedures within this website to assess exoplanet habitability
2. We can follow one of two equations depending on the properties we'll be using for the analysis

The first equation uses the flux of the star the planet's orbiting–basically the amount of energy the star is emitting in a given area per second)–which relates to the planet's surface temperature, the goldilocks zone, etc., and the radius of the planet which partially relates to its mass and density and thus gravity.

Mathematically, it subtracts the quadratic mean of the difference to sum ratios of the fluxes and radii from 1. Essentially, it quantifies the individual differences between the properties on a range from 0 to 1 before taking their quadratic mean (squaring each of them, dividing by the number of properties involved, and then taking the square root of everything). This ensures that if the differences between the properties is negligible, the planet's ESI assessment will be closer to 1 (Earth)

The second equation allows us to define our variables, and is one of the original equations for the ESI. It's similar to the first equation, but instead of taking the quadratic mean of the differences, it takes the geometric mean (multiply n property differences and then take the nth root of it all)

In addition, the second equation's documentation also provides a table of parameters to define the ESI, mainly the properties used, and the weights (importance) assigned to each. This proves helpful for us, as it sets a baseline of standardization.

Planetary Property	**Reference Value**	**Weight Exponent**
Mean Radius	1.0 Eu	0.57
Bulk Density	1.0 Eu	1.07
Escape velocity	1.0 Eu	0.70
Surface Temperature	288 K	5.58

Eu means Earth Unit

Both equations work out relatively the same, so which one do we use?

Recall that a research plan/idea must be feasible within our tools/restrictions. However, a key distinction is that this isn't an either/or choice. In fact, the fact that the two equations cover different types of observables allows us to use both in the instance where one equation can't work. Thus, in order to work towards a plan, we must first analyze each equation, and come up with a procedure for our analysis. We will save this for our next lesson.

ASSIGNMENTS:

- Take a look at the website provided. Make note of important details within using the ESI. Going back to our open-ended question, what should we be looking for now?
- Each equation has its own set of exoplanet properties involved. However, some properties must be derived from simpler observable properties–these are what are known as high-level observables. Do some online research on the properties involved in each equation. Are they low-level or high-level? If they're high-level, list 3 lower-level observables used to derive them.

4. Breaking Down the Goals: Part 1

We narrowed our data analysis procedure down to using two equations within the Earth Similarity Index. Moreover, both equations can be used to spot each other. In this lesson, we will be taking a look at equation 1, and mathematically analyzing the properties involved within it. The goal of our mathematical analysis is to break any high-level observables (derived quantities from directly observed quantities) into lower-level observables that exist as actual properties we can access as listed within the Open Exoplanet Catalogue.

Mathematical Analysis

NOTE: While not recommended, if you do not wish to see the mathematical derivations behind everything, skip to page 17 for the summaries

Recall that equation 1 is as follows:

$$ESI(S,\ R)\ =\ 1\ -\sqrt{\frac{1}{2}\left[\left(\frac{S-S_{earth}}{S+S_{earth}}\right)^2\ +\ \left(\frac{R-R_{earth}}{R+R_{earth}}\right)^2\right]}$$

While the radius can be obtained via multiple methods of observation, the stellar flux is a high-level observable derived from multiple low-level observables. Stellar Flux is the amount of energy a star emits in a given area per second, and it diminishes with distance from the star (the energy density decreases). Note that this mathematical analysis will be referencing the Open Exoplanet Catalogue's README.md Data Structure table.

Stellar Flux is mathematically defined as:

$$F\ =\ \frac{L}{4\pi d^2}$$

Where L is the star's intrinsic luminosity (total amount of energy emitted per second), d is the distance from which the star is observed (in our case it's the planet), and F is stellar flux.

Note that in the ESI, flux is denoted by S–it is common for physics variable letters/symbols to vary, and thus it is important to understand what each symbol is denoting based on the context.

Another important thing to note is that luminosity (L) is also a high-level observable on its own. Luminosity is mathematically defined as:

$$L\ =\ \sigma A T^4\ =\ 4\pi R^2 \sigma T^4$$

Where A is the surface area of the star ($A = 4\pi R^2$), R is the radius of the star, T is the temperature of the star (surface temperature), and σ is the Stefan-Boltzmann constant,

$$\sigma \approx 5.67 \times 10^{-8}\, Wm^{-2}K^{-4}$$

With this, we can write stellar flux in terms of low level observables:

$$S = \frac{4\pi R^2 \sigma T^4}{4\pi d^2} = \frac{R^2 \sigma T^4}{d^2}$$

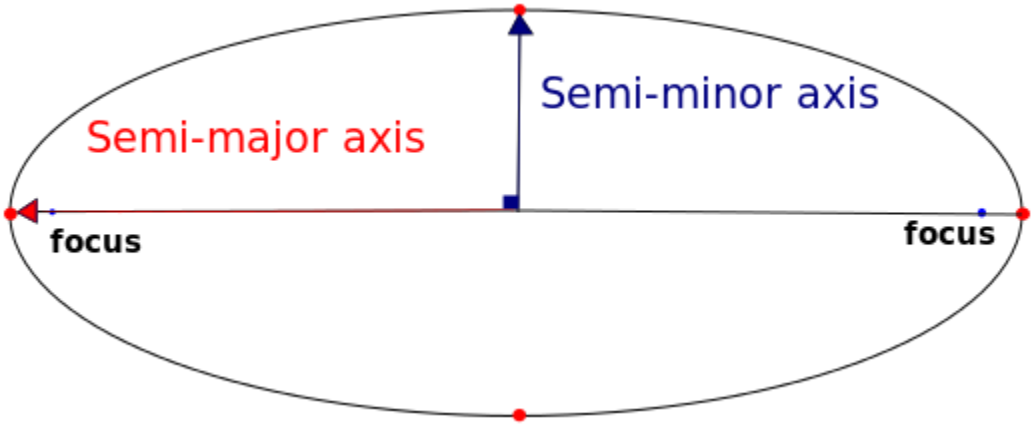
We have successfully broken down the stellar flux experienced by an exoplanet into low-level observables based on the exoplanet system.

CONCEPT CHECK: Using our derivations above, which low-level observables are needed in order to complete our ESI equation?

ANSWER: Stellar Radius, surface temperature of the star, distance from exoplanet to star, and exoplanet radius

HOWEVER, THERE IS A PROBLEM: The distance from a planet to a star isn't a low-level observable most of the time.

According to Kepler's first law of planetary motion, the orbits of planets around stars are elliptical. Simply, they're not perfect circles, so their distance to their stars always changes. We can use the semimajor axis (a), which is half the length of the longest diameter of an ellipse, and which serves as the average distance from the exoplanet to the star.



However, the semi-major axis is not always obtainable. Sometimes, it must be derived from other quantities. In order to derive the semi-major axis, we use Kepler's Third Law in most cases, which is mathematically defined as:

$$P^2 = \frac{4\pi^2}{G(m_1+m_2)}a^3$$

Where P is the orbital period (time it takes for the planet to complete a full orbit) in seconds, a is the semimajor axis in meters, m is used to denote the masses of either the star or the planet, and G is Newton's gravitational constant, $G \approx 6.674 \times 10^{-11} Nm^2kg^{-2}$

Here, we can rearrange the equation to solve for the semimajor axis:

$$a = \left(\frac{P^2G(m_1+m_2)}{4\pi^2}\right)^{\frac{1}{3}}$$

Recall that a fractional exponent is the same thing as a root

It can get even more complicated here. The orbital period is also a derived quantity from exoplanet analysis, depending on the methodology used. For now, we won't go into too much detail on how the orbital period can be found, and we can save that for a more detailed discussion on our procedure

General Needed Quantities:

- Stellar Radius
- Surface Temperature of Star
- Semi-major axis/distance from exoplanet to star
    - Orbital period (if needed)
- Exoplanet Radius

All of these quantities exist as tags within the Open Exoplanet Catalogue. We can come up with approximations if we have missing data once we take a look at the data itself, but for now, these are the primary variables needed for equation 1.

ASSIGNMENTS:

- Look up one way astronomers can determine the properties of a star (radius, mass, temperature), and take note of it.

5. Breaking Down the Goals: Part 2

In the last lesson, we broke down equation 1 of the Earth Similarity Index into physical quantities that are present in the Open Exoplanet Catalogue. In this lesson, we will do the same for Equation 2.

Mathematical Analysis

Recall that equation 2, in our context, uses mean radius, bulk density, escape velocity, and surface temperature as the exoplanet properties of comparison. Let's break each of these down:

- Mean Radius - Average Radius of the Exoplanet (given in OEC)
- Bulk Density - Average Density of the Exoplanet (derived)
- Escape Velocity - The velocity required to escape a planet's gravity (derived)
- Surface Temperature - Surface Temperature of Planet (TRICKIEST)

Let's take a closer look at our derived quantities, starting with density. Physically speaking, the density of an object is the amount of mass present in a given volume of said object. Mathematically, the bulk density of an exoplanet is defined as:

$$\rho = \frac{M}{V}$$

Where ρ *is the bulk density,* M *is the mass of the planet, and* V *is the volume*

Volume is not a valid property in the exoplanet catalogue. Planets are rarely perfect spheres, but we can estimate them to be so. The volume of an exoplanet can be obtained via:

$$V = \frac{4}{3}\pi R^3$$

Where R *is the mean radius of the planet*

Thus, the equation for the bulk density of an exoplanet can be rewritten as:

$$\rho = \frac{3M}{4\pi R^3}$$

Now let's take a look at escape velocity. The escape velocity can be derived with the same quantities used to derive the bulk density, and is defined as:

$$v_{escape} = \sqrt{\frac{2GM}{R}}$$

Where M is the exoplanet mass, R is the radius, and G is Newton's Gravitational Constant,

$$G \approx 6.674 \times 10^{-11} Nm^2 kg^{-2}$$

Note that this is the velocity required to escape the planet assuming you start at its surface, which is why we use the radius. We can see how with the addition of mass, we can use the mean radius of a planet to find all of these other properties.

Surface temperature is arguably the trickiest exoplanet property to find, and is also unfortunately one of the most important when assessing habitability, as it strongly affects whether or not life will be able to survive on a planet. As of now, direct surface temperature measurements of planets are incredibly rare, and can only be found through varying approximations. We will discuss this in our next lesson when we take a look at the data itself to see which approximations we need.

General Needed Quantities:

- Exoplanet Radius
- Exoplanet Mass
- Surface Temperature

Thus, we have determined the general quantities we need in order to conduct experimentation and determine the ranking of planets on the ESI scale within the OEC.

ASSIGNMENTS:

- Google the mass of Earth and its radius, and try deriving both its bulk density and escape velocity at its surface. Be careful with units!
- Why do you think exoplanet surface temperatures are so hard to find, especially for exoplanets at large distances?

6. Constraining the Methodology

In order to come up with a solid methodology for your analysis, it may be necessary to first play around with the data and tools available to you. In our case, it will be data within the Open Exoplanet Catalogue

Understanding the Data Structure

One of the trickiest requirements for data analysis is understanding how the data is formatted and how data points within the set relate to each other. This is especially important in cases like exoplanet studies, where matching the wrong planet to the wrong star can have drastic impacts on prediction quality.

One of the strongest ways to determine dataset relationships is using the len() command in combination with one's understanding of the dataset itself. The len() command provides the length of a given list within the parentheses. Let's first list out what we know:

WHAT WE KNOW:

- The dataset contains planetary systems, stars, and planets
- Multiple planets can orbit one star, there can exist binary star planetary systems, and planets don't necessarily need to have a star to be in the catalog

Using the len() command on each category (systems, stars, planets) allows us to understand how each of them work.

CONCEPT CHECK: Should there be more or less planets than systems? Should there be more or less stars than systems? Use your knowledge of exoplanet detection to answer this question.

Combining the len() command and the print() command, along with an f" string that allows us to insert commands within printed text, we can conveniently represent the lengths of all our most important data categories:

```
# Let's take a look at the structure of the data with the len() command.

print(f'Number of Systems: {len(oec.findall('.//system'))}')
print(f'Number of Exoplanets: {len(oec.findall('.//planet'))}')
print(f'Number of Stars: {len(oec.findall('.//star'))}')
```

```
Number of Systems: 4081
Number of Exoplanets: 5414
Number of Stars: 4300
```

The next important question is whether or not the star and exoplanet categories each correspond to specific stars and exoplanets within these given systems. We can verify by following the third for loop in the tutorial code, which uses the .findall feature on each individual system.

```
for system in oec.findall(".//system"):
    print(len(system.findall(".//star")), len(system.findall(".//planet")))
```

This ends up printing the number of stars and exoplanets in each system. We want the total number of stars and exoplanets across all systems. We'll make a few modifications to the code to do so.

The first thing we can do is create an empty array [ ]. Arrays (or lists) have a neat function called .append() that lets you add an element of desire to the array. The append function is crucial to all data analyses that involve processing data. We can obtain the length of the stars of each system like we did earlier, and then append them all to an empty array, before adding up all the star counts within said array to get the total number of stars. We'll do the same for exoplanets.

```
# Let's verify that stars and exoplanets correspond to the stars and exoplanets within the systems

star_array = []
planet_array = []

for system in oec.findall(".//system"):
    star_array.append(len(system.findall('.//star')))
    planet_array.append(len(system.findall('.//planet')))

print(f'Number of Stars in Systems: {sum(star_array)}') # The sum() command adds up all numbers
within the array
print(f'Number of Planets in Systems: {sum(planet_array)}')
```

```
Number of Stars in Systems: 4300
Number of Planets in Systems: 5414
```

We have now verified that the exoplanets and stars correspond to their systems. Thus, from now on, we can access the exoplanet and star data within the .//system column of the dataset

CONCEPT CHECK: Why do we have to access exoplanet and star data from the exoplanet systems, and not the exoplanets and stars directly?

ANSWER: Our ESI calculations for exoplanets relies on both the exoplanet's properties and its host star's properties. We have to assign each exoplanet to a star, assuming a single-star planetary system.

Searching for Analysis Limitations

As mentioned earlier, due to limitations within our current detections and studies of exoplanets, many exoplanets have missing data points. In this portion of the lesson, we will determine how much available exoplanet data we have for each needed quantity for analysis.

Equation 1 Quantities:

- Stellar Radius
- Surface Temperature of Star
- Semi-major axis/distance from exoplanet to star
    - Orbital period (if needed)
- Exoplanet Radius

Equation 2 Quantities:

- Exoplanet Radius
- Exoplanet Mass
- Surface Temperature

Lesson notes regarding the code will be in the code annotations:

```
'''
Let's check how much missing data we have for each property of interest

As seen in the tutorial code, properties of given objects are accessed via .findtext().
Moreover, missing data points for a given object are represented by "None"

To make things easier, we'll start by creating a function, which is basically a line of code that
gets run every time we call it.
'''


def calculate_availabledata(data_array):
  '''
  Functions are structured as def functionName(args):
  Args (arguments) are variables that the function defines itself, and that you can pass to the
function
  For example, in this case, you can pass an array of properties into the function for it to work on
```

```
  '''

  available_data = list(filter(None, data_array)) # We can use the filter command to filter out any
missing data within all the star properties

  return len(available_data), len(data_array) # If you store a function within a variable, you can
use the outputs of this return command as variables themselves

# For simplicity's sake, we'll run this on each individual star & exoplanet without checking if
entire systems have the properties
# We'll check if both an exoplanet and a star have the needed properties once we begin analysis

# Let's create a list of names for the quantities we want to use. They'll be used as variables
within a for loop

star_quantities = ['radius', 'temperature']
planet_quantities = ['semimajoraxis', 'radius', 'mass', 'temperature']

# Let's get the metrics of the star quantities first

for quantity in star_quantities: # Quantity serves as our variable here, representing quantity names

  star_data = [] # Store star data within a temporary array for this quantity

  for star in oec.findall('.//star'): # Nested loop looping through each star now.

    star_data.append(star.findtext(quantity)) # We can use our property name as a substitute in
findtext

  available, total = calculate_availabledata(star_data) # We can set multiple variables w/ commas

  print(f'Stars that have {quantity} defined: {available}/{total}')
  print(f'Stars that do not have {quantity} defined: {(total-available)}/{total}')

# Do the same for planets

for quantity in planet_quantities:

  planet_data = []

  for planet in oec.findall('.//planet'):

    planet_data.append(planet.findtext(quantity))

  available, total = calculate_availabledata(planet_data)

  print(f'Planets that have {quantity} defined: {available}/{total}')
  print(f'Planets that do not have {quantity} defined: {(total-available)}/{total}')
```

OUTPUT:

```
Stars that have radius defined: 3598/4300
Stars that do not have radius defined: 702/4300
Stars that have temperature defined: 3665/4300
Stars that do not have temperature defined: 635/4300
Planets that have semimajoraxis defined: 2813/5414
Planets that do not have semimajoraxis defined: 2601/5414
Planets that have radius defined: 4166/5414
Planets that do not have radius defined: 1248/5414
Planets that have mass defined: 2777/5414
Planets that do not have mass defined: 2637/5414
Planets that have temperature defined: 1636/5414
Planets that do not have temperature defined: 3778/5414
```

It can be seen that most stars have a radius and temperature property. As for planets, the existence of a semimajor axis is almost 50/50, most planets have radius defined, mass is 50/50, and most planets don't have a defined temperature.

Due to the fact that we require systems that match all of these properties, we should most likely find approximation fallbacks for the following properties:

- Planet semimajor axis
- Planet temperature

ASSIGNMENTS:

- Answer the following questions:
    - What are some important considerations when we do choose approximations for specific quantities?
    - What happens if the approximations fail? What do we do?
    - Why do you think planetary habitability assessment is so hard, even when a majority of the data isn't missing for most stars and planets? What did we not check during this data assessment?

7. Finalizing a Research Plan

In the previous lessons, we have broken down aspects of our proposed analysis plan, and also defined constraints to our data analysis as a whole. In this lesson, we will propose a well-rounded, analysis-ready research plan.

Defining the Research Question

Every research plan requires a research question that serves as the basis for why the research is being conducted in the first place. While some research questions may come naturally, usually the most well-rounded and specific research questions come after extensive literature review and project assessment.

Good research questions are defined by good research ideas. Let's revisit the guidelines we laid out in our first lesson:

A GOOD RESEARCH IDEA IS:

- Feasible given tools, restrictions, and time constraints
- Grounded in theory/principle/logic
- Methodologically and analytically consistent with research practices
- Focused, both topic-wise and goal-wise

Let's add a new guideline with research questions in mind:

- Working to address or elaborate on a specific problem or subject within the discipline

CONCEPT CHECK: Recall our analysis planning, our introduction to habitability, the overview and topic of the lesson, and our constraints. Following the guidelines, propose a research question to serve as the basis for the project

While research questions may vary per person, we define this research question for the sake of teaching the course:

How can the assessment of exoplanet habitability via the Earth Similarity Index be carried out in the presence of missing data?

Setting Goals

In order for a plan to have a direction, goals must be defined. In our case, we have already discussed goals in the past, specifically for analysis.

GOALS:

- Obtain sufficient planetary system data to assess the habitability of an exoplanet through one of the two ESI equations
- Discover approximations via given exoplanet properties that can serve as substitutes for missing data
- Identify uncertainties within the analysis
- Visualize and analyze the results of the analysis**

Determining the Course of Action

We have the following resources:

- Google Colab
- Open Exoplanet Catalogue
- Earth Similarity Index

Given our goals, we can easily design an experiment for carrying out our research. In the case of coding projects, most researchers tend to first define problems to solve within the code, and then elaborate on how said problems can be solved whilst coding. The experimental design of the overall coding will be covered in the next lesson.

CONCEPT CHECK: Design a 4-6 step overview plan for the project

1. Load the Open Exoplanet Catalogue
2. Design code for selection of systems and also approximations for missing data
3. Select only the systems with enough data to use the ESI
4. Identify areas of possible uncertainty within the project using knowledge of exoplanets
5. Visualize the distributions of habitability with respect to individual variables involved within the equations (each equation will have differing representation capabilities)

ASSIGNMENTS:

- Look up ways physicists/astrophysicists can visualize large amounts of data. Summarize the methodology of at least three
- Look up metrics physicists/astrophysicists use to assess the quality of data. Explain at least two
- Look up common Python libraries astrophysicists use for data analysis. Find at least three. Go onto their Application Programming Interface (API) websites, which provide the documentation for each command, and be able to explain two commands for each.

8. Structuring the Code

In order to carry out analysis, we must first get an idea of what we want our code to look like. This lesson will provide tips on structuring coding projects in general, and ultimately create a structure for the analysis to be conducted within this class.

Fetching Necessary Tools

In order to begin working on a coding project, we must first collect all the necessary tools we need to code. By tools, we mainly refer to either resources, software, and in the case of Python, libraries. Some tools may be added mid-analysis; however, it is usually standard to collect any common tools used in analyses.

We already have the following tools:

- Open Exoplanet Catalogue
- Google Colab Notebook

In order to perform data analysis, astrophysicists use a variety of Python analysis libraries. Python is central to a majority of data analysis research projects due to its ability to access powerful analysis tools.

From a quick internet search, we can find that the most common Python libraries astrophysicists use tend to be:

- NumPy (calculations)
- Astropy (constants & tools)
- Matplotlib (graping/plotting capabilities)

We will be using these within our analysis. Recall that the complete functionality of a library can simply be accessed by running "import [library]" at the start of the script.

Analysis Setup

Similar to how we're going to import tools for analysis, we can also create our own tools to use later on. In coding, these come in the form of functions, lines of code that can be called on and run anytime once defined. Functions come in handy when you need to reduce clutter in your code and help reduce the work required to code. Recall that functions can be defined using "def [function_name]([anything you want to give the function to use]):"

CONCEPT CHECK: What are some things in this project we need to create functions (pre-defined tools) for?

As a preliminary list, we require functions for the following tasks:

- Applying selection cuts to data
- Calculating higher-level observables (e.g. escape velocity)
- Approximating Missing Data
- Calculating a planet's Earth Similarity
- Post-assessment analysis

It should be understood that since we're using a notebook where code can be tested fast, there is less of a need for functions. However, in larger projects that involve complete scripts, functions are a must-have. Moreover, we can add data approximations afterwards once we finish a test-run of our data processing–code doesn't need to work perfectly the first try; in fact, it never does.

Designing Data Processing Methodology

Let's design a procedure for processing the data. Since we already know our tasks, this will be fairly easy.

PROCEDURE:

1. Load OEC
2. Loop through a list of defined properties to find
3. For each property, loop through every planetary system, adding said property to a storage (usually a list or dictionary) for planetary system properties
4. Apply selection cuts to each system to work with ESI equation 1, using equation 2 if not possible.
5. Calculate the ESIs of each planet within each planetary system.
6. For properties lacking data, add approximations
7. Note the use of approximations for future analysis

Visualizing Data

Data can be arguably understood 100x easier if visualized. Most physicists use histograms to visualize data, which represent the distributions of the data–representing how many planets there are with a specific temperature, for example. Histograms can either be 1D or 2D, but we will go into depth on this in the next lesson. For now, we can just understand that we'll be using matplotlib to relate ESI calculations and various exoplanet and star properties. Further data analysis will be detailed in the actual coding portion of this class

ASSIGNMENTS:

- Draw a visual representation of how data processing will be done for this project

9. Code Setup

This lesson will serve as a straight-to-the-point guide to setting up code in order for us to perform analysis. Code explanations will be within the annotations, while explanations of the theoretical motivations will be presented here.

Imports

We will need to import a few external Python libraries for data analysis during this project. Each of these have been mentioned within the course:

```
'''
Let's start by importing external libraries
'''

import xml.etree.ElementTree as ET, urllib.request, gzip, io # OEC Access

import numpy as np # Number Operations
import matplotlib.pyplot as plt # Plots & Graphs
from astropy.constants import G, sigma_sb, M_earth, R_earth, GM_earth L_sun, au # Astropy gives us
the solar system's constants
```

The Astropy Constants imported are as follows:

- G: Newton's Gravitational Constant
- sigma_sb: Stefan Boltzmann Constant
- M_earth: Earth mass
- R_earth: Earth Radius
- GM_earth: G * M_earth
- M_jup: Jupiter mass (mass is defined in jupiter masses in the OEC)
- L_sun: Luminosity of Sun
- AU: The semimajor axis of Earth, formally defined as an Astronomical Unit [AU]

Project Variables

Each project has variables that it will reference throughout the code. It's important to set these variables early in order to stay organized.

For this project, we will only need the url we're trying to access and the properties we want to use.

```
'''
Project Variables
'''
```

```
oec_url = "https://github.com/OpenExoplanetCatalogue/oec_gzip/raw/master/systems.xml.gz" # If we
want to use a different data format, we can change the URL

# Names in line with the tag names in the OEC
system_properties = ['distance']
planetary_properties = ['discoverymethod', 'mass', 'radius', 'semimajoraxis', 'temperature',
‘period’]
star_properties = ['temperature']
```